

Contents

Empathy-in-Action Probes V2: Onboarding Document 1

Project Overview 1

Current Status 1

Open-Source Models (5 Families) 2

Your Tasks 2

Compute Access: Lambda Labs 3

Code Structure 4

Technical Notes 5

Communication 6

Key Deliverables 6

Questions? 6

Empathy-in-Action Probes V2: Onboarding Document

Project Lead: Juan Cadile **Collaborator:** Maxi **Date:** December 2024 **Target:** NeurIPS / FAccT submission

Project Overview

We’re building interpretability tools to detect and steer “empathy-in-action” in large language models. The core idea:

- 1. **Detection:** Train linear probes on model activations to classify empathic vs non-empathic responses
- 2. **Steering:** Use the learned direction to modify model behavior at inference time
- 3. **Mechanistic Analysis:** Understand *how* models represent empathy (which layers, attention heads, circuits)

Key Research Questions

- Is empathy-in-action linearly represented in LLMs? (Hypothesis: AUROC > 0.9)
- Does the representation transfer across model families?
- Can we steer models to be more/less empathic?
- Which layers causally mediate empathic behavior?

Current Status

Dataset: COMPLETE

Model	Type	Pairs	Status
Claude Sonnet	Closed	500	Done
Claude Haiku	Closed	500	Done
GPT-4o	Closed	500	Done
GPT-5.1	Closed	500	Done

Model	Type	Pairs	Status
Gemini 2.5 Flash	Closed	500	Done
Llama-70B	Open	500	Done
Gemma-27B	Open	500	Done
Qwen-32B	Open	500	Done
Yi-34B	Open	500	TODO
Mistral-24B	Open	500	TODO

Location: data/contrastive_pairs/

Each pair contains: - A scenario (e.g., “friend going through divorce”) - An empathic response - A non-empathic response - Metadata (model, scenario type, etc.)

Open-Source Models (5 Families)

These are the models you’ll work with for interpretability experiments:

Model	Params	Family	HuggingFace ID
Llama-3.1-70B	70B	Meta	meta-llama/Llama-3.1-70B-Instruct
Gemma-2-27B	27B	Google	google/gemma-2-27b-it
Qwen-2.5-32B	32B	Alibaba	Qwen/Qwen2.5-32B-Instruct
Yi-1.5-34B	34B	01.AI	01-ai/Yi-1.5-34B-Chat
Mistral-Small-24B	24B	Mistral	mistralai/Mistral-Small-3.1-24B-Instruct-250

Important: Use BF16 precision for all experiments (NOT FP8 or 4-bit quantization).

Your Tasks

All tasks are tracked as GitHub issues: <https://github.com/juancadile/empathy-probes/issues>

Phase 1: Data Preparation (Priority: HIGH)

Issue	Task	Est. Hours
#19	Generate pairs for Yi-34B + Mistral-24B	8-12h
#2	Build unified dataset (merge all JSONLs)	2-3h
#3	Write activation extraction script	4-6h

Phase 2: Core Experiments (Priority: HIGH)

Issue	Task	Est. Hours
#4	Train linear probes on all 5 models	6-8h
#5	Cross-model probe transfer (5x5 matrix)	4-6h
#6	Random baseline validation	2-3h
#7	EIA behavioral correlation	3-4h

Phase 3: Steering (Priority: HIGH)

Issue	Task	Est. Hours
#8	Comprehensive steering experiments	25-35h
#9	Steering analysis & visualization	4-6h
#10	Cross-model steering transfer	8-10h

Phase 4: Mechanistic Interpretability (Priority: MEDIUM)

Issue	Task	Est. Hours
#11	Activation patching	10-15h
#12	Attention head / MLP ablation	8-10h
#13	SAE training (optional but high impact)	20-40h
#14	Path patching (optional)	10-15h

Phase 5: Scaling Analysis (Priority: MEDIUM)

Issue	Task	Est. Hours
#15	Add Gemma-2B, Gemma-9B for scaling curve	4-6h
#16	Scaling law analysis & figure	2-3h
#17	Cross-family geometric analysis	4-6h

Phase 6: Figures (Priority: LOW - Juan will handle)

Issue	Task
#18	Regenerate all figures for publication

Compute Access: Lambda Labs

You'll have access to a GH200 instance (96GB VRAM).

SSH Access

```
# SSH key will be provided separately
ssh -i /path/to/key.pem ubuntu@<INSTANCE_IP>
```

```
# Project directory
cd ~/empathy-probes
```

Environment Setup

```
# Create conda environment
conda create -n empathy python=3.11
conda activate empathy

# Install dependencies
pip install torch transformers vllm accelerate
pip install scikit-learn numpy pandas matplotlib seaborn
pip install einops safetensors

# Optional: SAE training
pip install sae-lens
```

Running Experiments

```
# 1. Generate contrastive pairs (Yi-34B example)
python src/generate_opensource_vllm.py --model yi-34b

# 2. Extract activations
python src/extract_activations_opensource.py --model llama-70b --all-layers

# 3. Train probes
python src/probe_extraction_cross_model.py --model llama-70b

# 4. Run steering
python src/steering_comprehensive.py --model llama-70b --layers 60,65,70
```

Code Structure

```
empathy-probes/
  data/
    contrastive_pairs/    -> Generated pairs (JSONL)
    activations/          -> Extracted activations (.pt)
    scenarios/            -> Scenario definitions
  src/
    generate_opensource_vllm.py    -> Pair generation
    extract_activations_opensource.py -> NEW (you write this)
    probe_extraction_cross_model.py -> Probe training
    steering_comprehensive.py      -> Steering experiments
```

activation_patching.py	-> NEW (you write this)
component_ablation.py	-> NEW (you write this)
results/	
probes/	-> Trained probe directions (.npy)
steering/	-> Steering results (.json)
figures/	-> Generated figures
notes/	-> Documentation

Technical Notes

Memory Management for 70B Models

```
# Use gradient checkpointing
model.gradient_checkpointing_enable()

# Or extract layer-by-layer
for layer_idx in range(num_layers):
    activations = extract_single_layer(model, texts, layer_idx)
    save_activations(activations, f"layer_{layer_idx}.pt")
    del activations
    torch.cuda.empty_cache()
```

Activation Extraction Pattern

```
def extract_activations(model, tokenizer, texts, layer_idx):
    activations = []

    def hook(module, input, output):
        # Mean pool over sequence length
        act = output[0].mean(dim=1) # [batch, hidden_dim]
        activations.append(act.detach().cpu())

    handle = model.model.layers[layer_idx].register_forward_hook(hook)

    with torch.no_grad():
        for text in texts:
            inputs = tokenizer(text, return_tensors="pt").to(model.device)
            model(**inputs)

    handle.remove()
    return torch.cat(activations, dim=0)
```

Steering Hook Pattern

```
def create_steering_hook(direction, alpha):
    def hook(module, input, output):
        # Add steering vector to residual stream
        output[0] = output[0] + alpha * direction.to(output[0].device)
```

```
    return output
return hook
```

Communication

- **GitHub Issues:** Use for task tracking and technical discussions
- **Slack/Discord:** TBD for quick questions
- **Weekly Sync:** TBD

Workflow

1. Pick an issue from GitHub (assign to yourself)
 2. Create a branch: `git checkout -b feature/issue-XX-description`
 3. Implement & test
 4. Push & create PR
 5. Juan reviews & merges
-

Key Deliverables

By the end of this collaboration, we need:

1. **Trained probes** for all 5 model families with AUROC metrics
 2. **5x5 transfer matrix** showing cross-model probe transfer
 3. **Steering results** with dose-response curves
 4. **Activation patching results** identifying causal layers
 5. **Scaling analysis** from 2B to 70B parameters
 6. **All figures** ready for publication
-

Questions?

Reach out to Juan for: - Access credentials - Research direction questions - Paper framing discussions

Good luck!